



中山大學
SUN YAT-SEN UNIVERSITY

函数和作用域 (1)

中山大学计算机学院



讲课人：潘茂林

目录

CONTENTS

01

函数的概念

02

函数的声明与定义

03

函数的调用

04

常用数学函数

05

变量的作用域



函数是什么

01

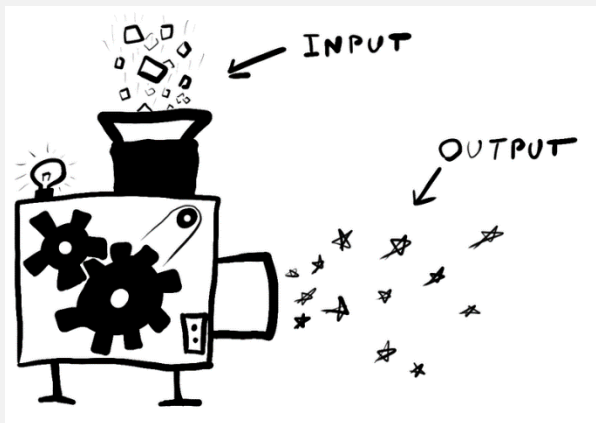
实现某个功能的**代码块**。
它是一个**黑盒子**

```
int add(int a, int b){  
    int c = a + b;  
    return c;  
}
```

一个实现 $a+b$ 功能的代码块

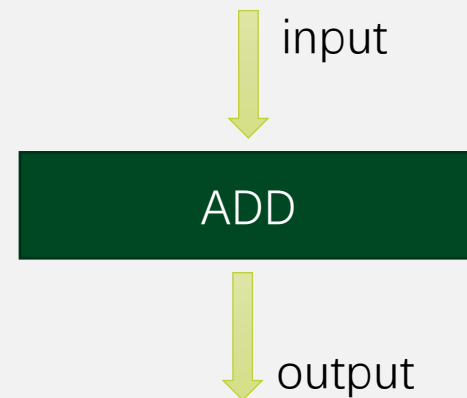
02

相当于一个机器，接受**输入**，
对输入进行**处理**，给出**输出**。



03

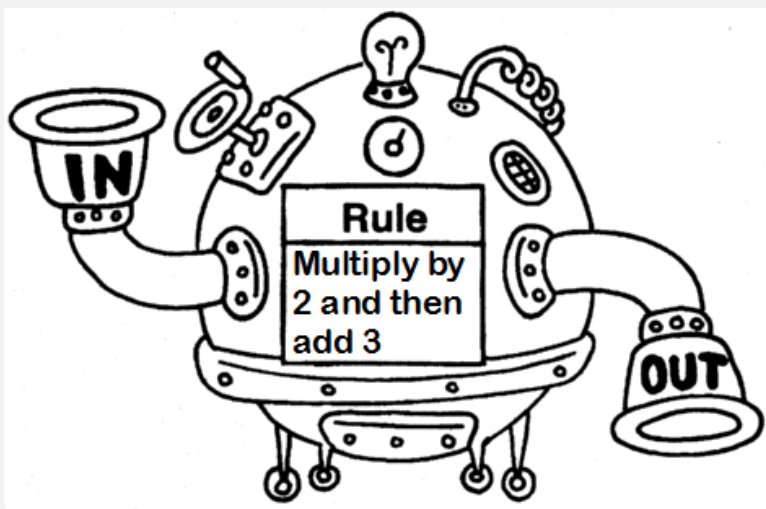
函数是一种抽象，它隐藏了算
法实现的细节





函数有什么用 – 构建程序的积木块

函数能够将一大块代码分解成若干个模块

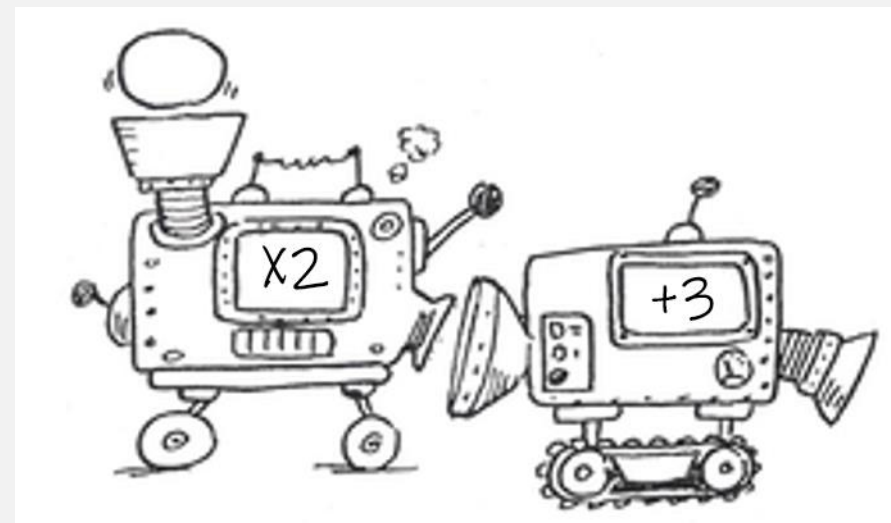
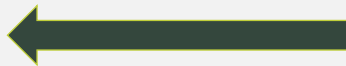


一个大程序

分解



组装



分解成两个小函数

易于理解!
易于调式!



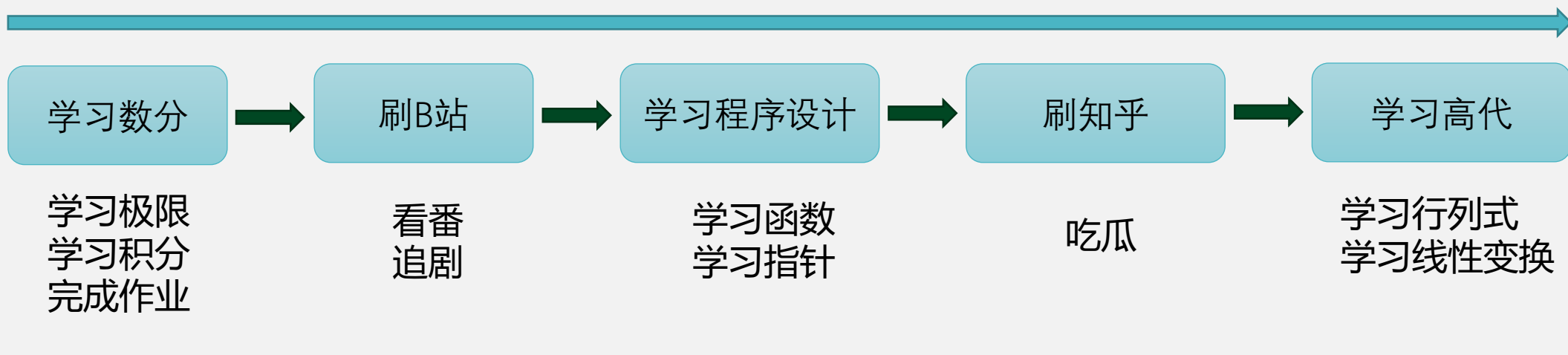
函数带来哪些好处

- 使代码可重用性!
- 不需要了解函数实现的细节, 易于理解
- 能够以**模块化**的形式编程
- **容易Debug!**

以**模块化**的形式对一天的生活进行规划!



时间





函数的声明与定义

函数的声明 (Declaration)

- 函数名 (function_name) ,
 输入参数 (parameter) ,
 输出类型 (return_type) ,
- 也叫函数的接口信息。

```
return_type function_name(parameter);
```

```
int add(int a, int b);
```

函数名: add
输入参数: 整型a, 整型b
输出类型: 整型
函数实现: 不需要知道

Q: 为什么需要声明?

A: 声明的意义是告诉编译器函数信息, 它在另一个地方写好了。虽然我可能不会编写它 (不明觉厉), 但先放函数声明在这里, 能用就好。

- 每次逐个声明函数很麻烦, 所以会按功能把一组函数声明放在一个头(.h)文件里, 方便使用。
- 分离了使用者和设计者的世界。是使用者和设计者之间共同遵守的**契约** (接口) 。
 - 作为使用者, 不关心函数是如何实现, 我们只关心如何使用函数, 有声明就足够了。
 - 作为设计者, 根据函数声明, 编写函数实现。例如: scanf 在不同C语言编译器中实现各不相同, 但使用这都可放心使用它们

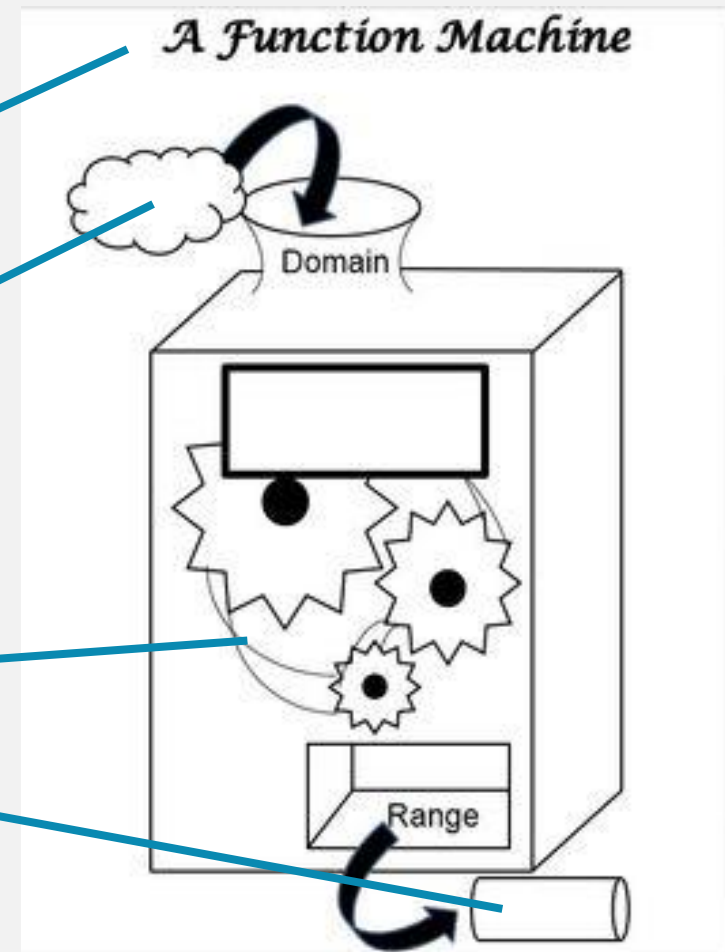


函数的声明与定义

函数的定义 (Definitions)

- 给出函数名 (function_name),
输入参数 (parameter),
输出类型 (return_type),
函数主体 (function body)。
- 函数主体是一个复合语句, 实现了函数的内部逻辑。

```
return_type function_name(parameter)
{
    function body;
}
```





函数的声明与定义

函数的定义 (Definitions)

- 给出函数名 (function_name) ,
 输入参数 (parameter) ,
 输出类型 (return_type) ,
 函数主体 (function body) 。
- 函数主体是一个复合语句, 实现了函数的内部逻辑。
 - **return** 关键字结束当前函数执行, 后面是函数输出的值

```
return_type  function_name(parameter)
{
    function body;
}
```

```
int add(int a, int b){
    int c = a + b;
    return c;
}
```

函数名: add

输入参数: 整型a, 整型b

输出类型: 整型

函数主体: c=a+b, 返回c

C语言中, 函数名必须是唯一的!



函数的声明与定义

函数的声明 (Declaration)

```
double __cdecl sin(double _X);
double __cdecl cos(double _X);
double __cdecl tan(double _X);
double __cdecl sinh(double _X);
double __cdecl cosh(double _X);
double __cdecl tanh(double _X);
double __cdecl asin(double _X);
double __cdecl acos(double _X);
double __cdecl atan(double _X);
double __cdecl atan2(double _Y, double _X);
double __cdecl exp(double _X);
double __cdecl log(double _X);
double __cdecl log10(double _X);
double __cdecl pow(double _X, double _Y);
double __cdecl sqrt(double _X);
double __cdecl ceil(double _X);
double __cdecl floor(double _X);
```

一目了然!
一眼知道如何调用!
舒心!

C语言自带的标准库函数

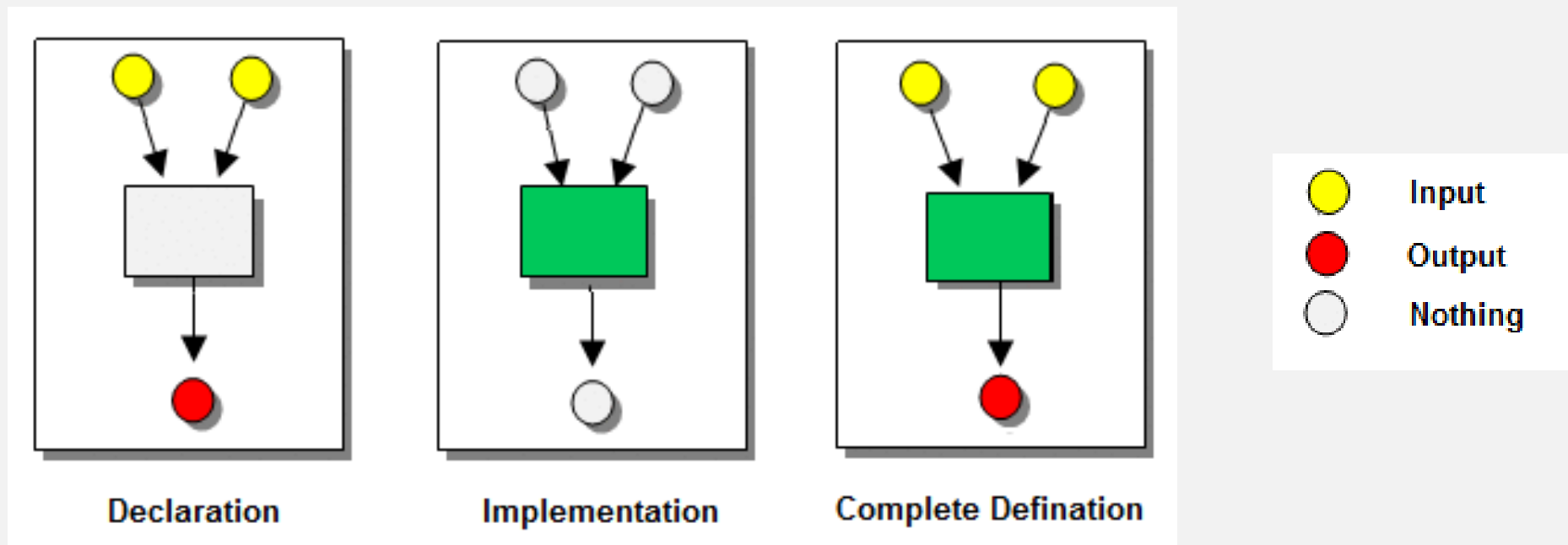
```
__mingw_ovr
__attribute__((__format__(gnu_scanf, 1, 2))) __MINGW_ATTRIB_NONNULL(1)
int scanf(const char *__format, ...)
{
    int __retval;
    __builtin_va_list __local_argv; __builtin_va_start(__local_argv, __format);
    __retval = __mingw_vfscanf(stdin, __format, __local_argv);
    __builtin_va_end(__local_argv);
    return __retval;
}

__mingw_ovr
__attribute__((__format__(gnu_scanf, 2, 3))) __MINGW_ATTRIB_NONNULL(2)
int fscanf(FILE *__stream, const char *__format, ...)
{
    int __retval;
    __builtin_va_list __local_argv; __builtin_va_start(__local_argv, __format);
    __retval = __mingw_vfscanf(__stream, __format, __local_argv);
    __builtin_va_end(__local_argv);
    return __retval;
}
```

不关心函数怎么实现
需要在万千代码中辛苦找到函数接口
烦心!

函数的声明与定义

函数的定义与声明



C语言至少有一个函数： **main函数**！



C 程序与函数 (Functions)

- C程序的基本结构
 - 程序由若干个函数构成
 - 必须有一个 main 函数
 - 函数不能重名
 - 函数**应该 (should be)** 先申明, 后实现
- 如右程序, 有三个函数
 - main
 - Square
 - Cube

请编译, 修改, 并运行程序 function-syntax.c

```
#include <stdio.h>

int Square( int );      // function declarations
int Cube( int );

int main() {
    printf( "The square of 3 is %d\n", Square(3));
    printf( "The cube of 3 is %d \n", Cube(3));
    return 0;
}

// function implementations
int Square(int n) {
    return n*n;
}

int Cube(int n) {
    return n*n*n;
}
```



函数的形参与实参

- 形式参数/形参 (Formal **parameter**)
 - 函数在**申明和定义时**用的参数
 - 参数定义时由于没有具体值，只是一个**符号**或可存储特定数值的空间
 - 申明时可仅申明参数类型
 - 定义时必须给出形式参数名称，便于函数体使用
- 实际参数/实参 (Actual **argument**)
 - 函数在**使用时**实际替代形式参数的**值**
 - 实际参数与申明的形式参数必须数量相同，类型兼容
 - 如果实参形参类型兼容但不一致，则按初始化规则发生**强制转换**

请修改 function-syntax.c 的实参为double，如3.14，并运行程序



函数的调用

函数根据参数和返回值可以分为四类：

无参数，无返回值

```
void add(void){  
    ...  
    ...  
}
```

void
空，无

输出类型为 **void**:
可以没有return语句
函数语句块执行结束，
函数结束

有参数，无返回值

```
void add(int a){  
    ...  
    ...  
}
```

无参数，有返回值

```
int add(void){  
    ...  
    ...  
    return 6;  
}
```

这两者类型
要一致

有参数，有返回值

```
int add(int a){  
    ...  
    ...  
    return 6;  
}
```



函数的调用

函数根据参数和返回值可以分为四类：

无参数，无返回值

```
void add(void){  
    ...  
    ...  
}
```

调用
call

返回
return

```
int main(void){  
    ...  
    add();  
    ...  
    return 0;  
}
```

有参数，无返回值

```
int main(void){  
    ...  
    add(5);  
    ...  
    return 0;  
}
```

调用
a=5

返回

```
void add(int a){  
    ...  
    ...  
}
```

无参数，有返回值

```
int add(void){  
    ...  
    ...  
    return 6;  
}
```

调用
call

返回
x=6

```
int main(){  
    ...  
    int x = add();  
    ...  
    return 0;  
}
```

有参数，有返回值

```
int main(void){  
    ...  
    int x = add(5);  
    ...  
    return 0;  
}
```

调用
a=5
返回
x=6

```
int add(int a){  
    ...  
    ...  
    return 6;  
}
```



函数的调用

函数根据参数和返回值可以分为四类：

无参数，无返回值

```
void add(void){  
    ...  
    ...  
}
```

```
int main(void){  
    ...  
    add();  
    ...  
    return 0;  
}
```

```
int main(void){  
    ...  
    add(5);  
    ...  
    return 0;  
}
```

有参数，无返回值

```
void add(int a){  
    ...  
    ...  
}
```

无参数，有返回值

```
int add(void){  
    ...  
    ...  
    return 6;  
}
```

```
int main(){  
    ...  
    int x = add();  
    ...  
    return 0;  
}
```

```
int main(void){  
    ...  
    int x = add(5);  
    ...  
    return 0;  
}
```

有参数，有返回值

```
int add(int a){  
    ...  
    ...  
    return 6;  
}
```

两者
类型一致



函数的调用

```
int main(void){  
    int y = 5;  
    int x = add(y);  
    return 0;  
}
```

```
int add(int a){  
    ...  
    a = 9;  
    return 6;  
}
```

a=5

x=6

调用add函数后
y=5 or 9 ?



函数的调用

```
int main(void){  
    int y = 5;  
    int x = add(y);  
    return 0;  
}
```

a=5

```
int add(int a){  
    ...  
    a = 9;  
    return 6;  
}
```

调用 add 函数时，形参 a 使用 y 初始化
调用 add 函数后，y 依然是 5！



函数的调用

```
int main(void){  
    int y = 5;  
    int x = add(y);  
    return 0;  
}
```

a=5

x=6

```
int add(int a){  
    ...  
    a = 9;  
    return 6;  
}
```

函数参数的按值传递 (Call By Value)

main函数里的变量y和add函数里的变量a不是同一个变量，尽管它们的值一样，add函数里无论对变量a作什么操作，不会影响到main函数里的变量y。



函数的调用

Q: 我想对变量a的修改能影响到变量y怎么办? 比如我想写一个函数swap(a,b)用来交换a, b这两个变量的值.....

A: 那就用到另一种传递方式, 叫引用传递 (Call By Reference)

```
int main(){  
    int a = 1, b = 2;  
    swap(&a, &b);  
    return 0;  
}
```

地址传递

指针类型

```
void swap(int *x, int *y){  
    int t = *x;  
    *x = *y;  
    *y = t;  
}
```

由于变量储存在计算机内存的某个地方, 我们用**地址**来描述。
函数形参用指针类型, 传入某个变量地址, 用 * 解引用运算获取对应变量
注意: *x 是左值



函数的调用 – 小结与练习

- 函数是一个初等表达式
 - 函数返回值的类型就是表达式的类型
 - 函数遇到 return 语句或函数块执行完毕函数表达式完成求值
 - 除了返回 void 类型，必须使用 return 语句返回一个值
- 函数实参与形参
 - 形参是传引用则实参必须是 “&左值表达式” 或 “地址”
 - 形参是传值是实参必须是类型兼容表达式
 - 实参与形参必须数量一致，类型兼容

请运行 call-byref-swap.c 体验传值和引用的区别



函数的调用 – 小结与练习

- 函数参数表达式求值顺序

```
/*function parameter expression*/  
#include <stdio.h>  
  
void testfun(int a, int b, int c){  
    printf("%d,%d,%d\n", a, b, c);  
}  
  
int main() {  
    int a = 0;  
    testfun(a++,a++,a++);  
}
```

请问程序输出是什么？ 未定义



常用数学函数 – 要求掌握

- 在标头 `<stdlib.h>` 定义
 - `int abs( int n );`
- 定义于头文件 `<math.h>`
 - `double fabs( double arg );`
 - `double exp( double arg );`
 - `double log( double arg );`
 - `double log10( double arg );`
 - `double pow( double base, double exponent );`
 - `double sqrt( double arg );`
 - `double sin( double arg );`
 - `cos, tan, asin, acos, atan`
 - `double ceil( double arg );`
 - `double floor( double arg );`



变量的作用域

变量的作用域 (scope), 指能够访问 (access) 该变量的**范围**, 可以是全局, 可以是函数, 可以是for循环, 也可以是任意用大括号{}括起来的代码块。

```
int main(void){  
  
    int y = 5;  
  
    int x = add(y);  
  
    return 0;  
}
```

在main函数里无法访问到add函数里的变量a, 但能访问到变量x。

在add函数里, 无法访问到main函数里的变量y, 但能访问到变量a。

```
int add(int a){  
    ...  
    a = 9;  
    return 6;  
}
```

变量x的作用域是main函数
变量a的作用域是add函数

变量的作用域

根据作用域的范围，变量可以分为三类

位置	种类
在函数或代码块里	局部变量 (local variables)
在所有函数外面	全局变量 (Global variables)
在函数参数列表里	形式参数 (Formal parameters)



变量的作用域：局部变量

局部变量在函数（块）内声明，仅在该函数（块）内能够访问到。

一个
代码
块

```
void add(void){  
    int a;  
    a = 1;  
    {  
        int b = 2;  
        b = a;  
    }  
    a = b; //error  
    {  
        int c = b; //error  
    }  
}
```

另一个代码块

其实函数和代码块都是由**大括号{}**包裹起来的

在该大括号内声明的变量**仅**在该大括号内可以访问

在语句或语句块内定义的变量，又称为**自动变量**。
当语句或语句块执行结束，所有自动变量被释放。

这里，第一个语句块之后，b已不存在，无法访问
类似，for 语句初始化部分定义的变量，for语句结束后也无法访问



变量的作用域：局部变量

局部变量在函数（块）内声明，仅在该函数（块）内能够访问到。

一个代码块

```
void add(void){  
    int a;  
    a = 1;  
    {  
        int b = 2;  
        b = a;  
    }  
    a = b; //error  
    {  
        int c = b; //error  
    }  
}
```

另一个代码块



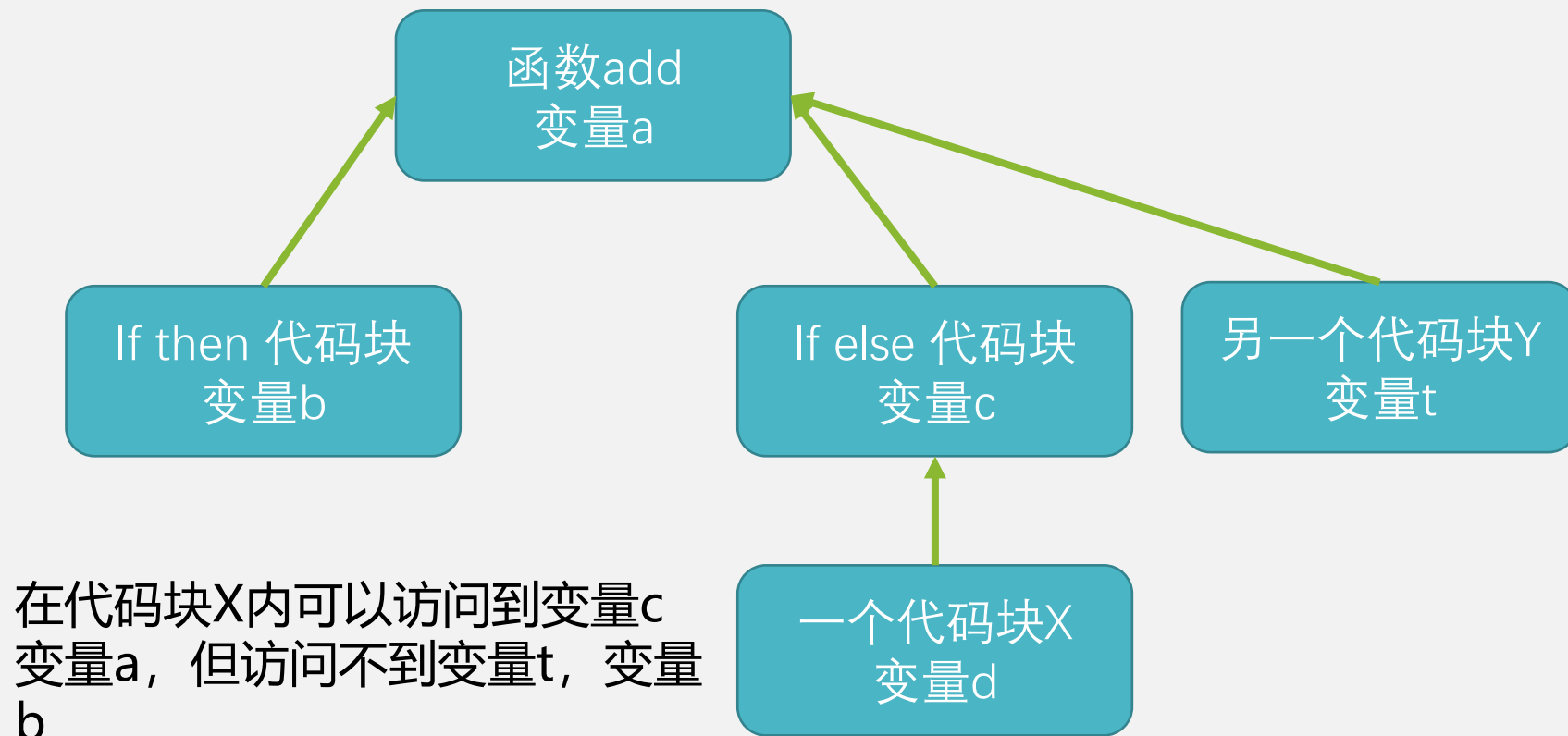
在一个域内，可以访问包含其域的块（父亲）内变量
局部变量的生命周期也仅在这个块内
一旦当前执行的代码离开了这个块，在该块的变量就被销毁



变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c = 2;  
        {  
            int d = a;  
        }  
        c = d; //error  
    }  
    a = 9;  
    {  
        int t = c; //error  
    }  
}
```

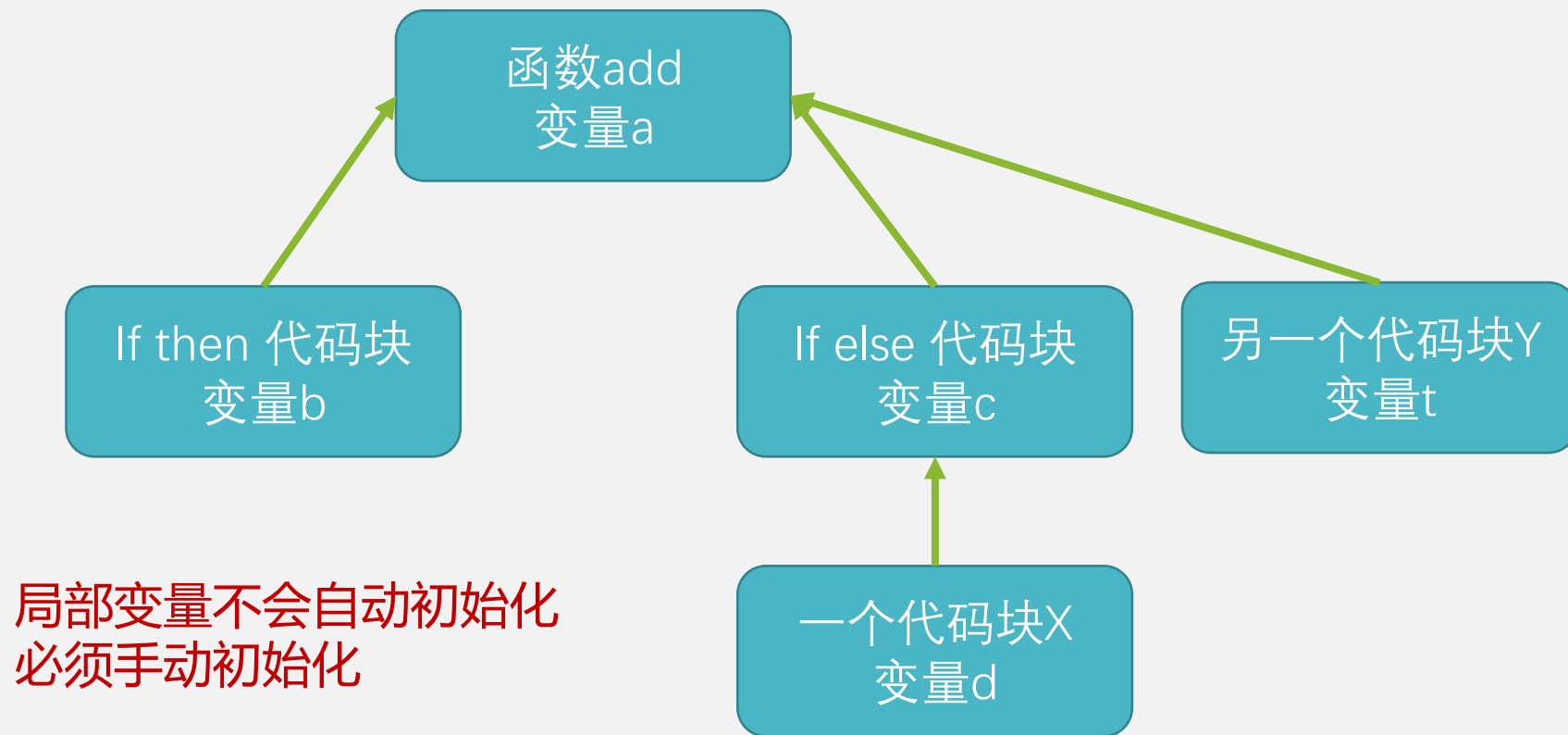




变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c = 2;  
        {  
            int d = a;  
        }  
        c = d; //error  
    }  
    a = 9;  
    {  
        int t = c; //error  
    }  
}
```



局部变量不会自动初始化
必须手动初始化

局部变量的生命周期仅在所在块内

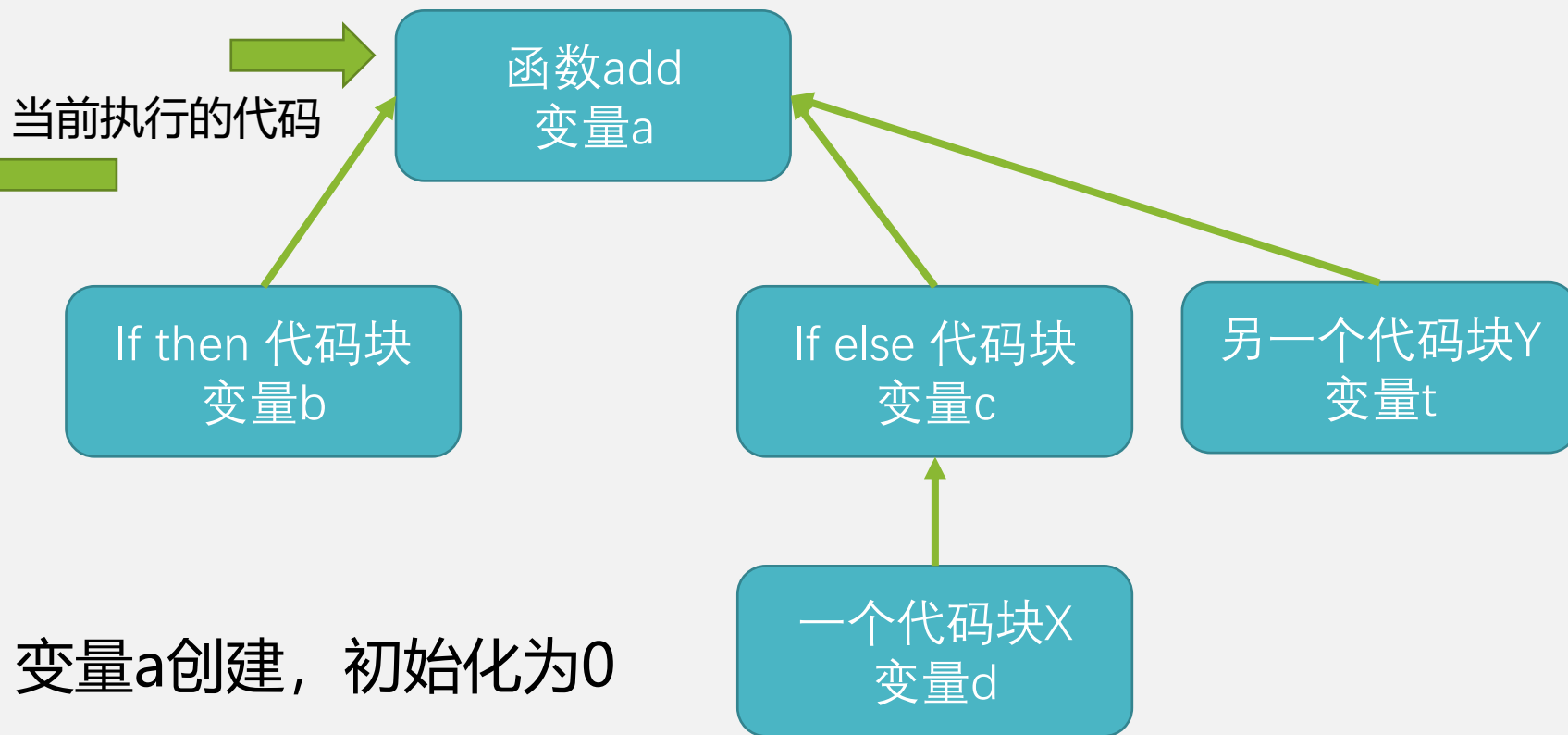
代码执行离开了该块，其变量均被销毁



变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c;  
        {  
            int d = a;  
        }  
        c = a;  
    }  
    a = 9;  
    {  
        int t = a;  
    }  
}
```

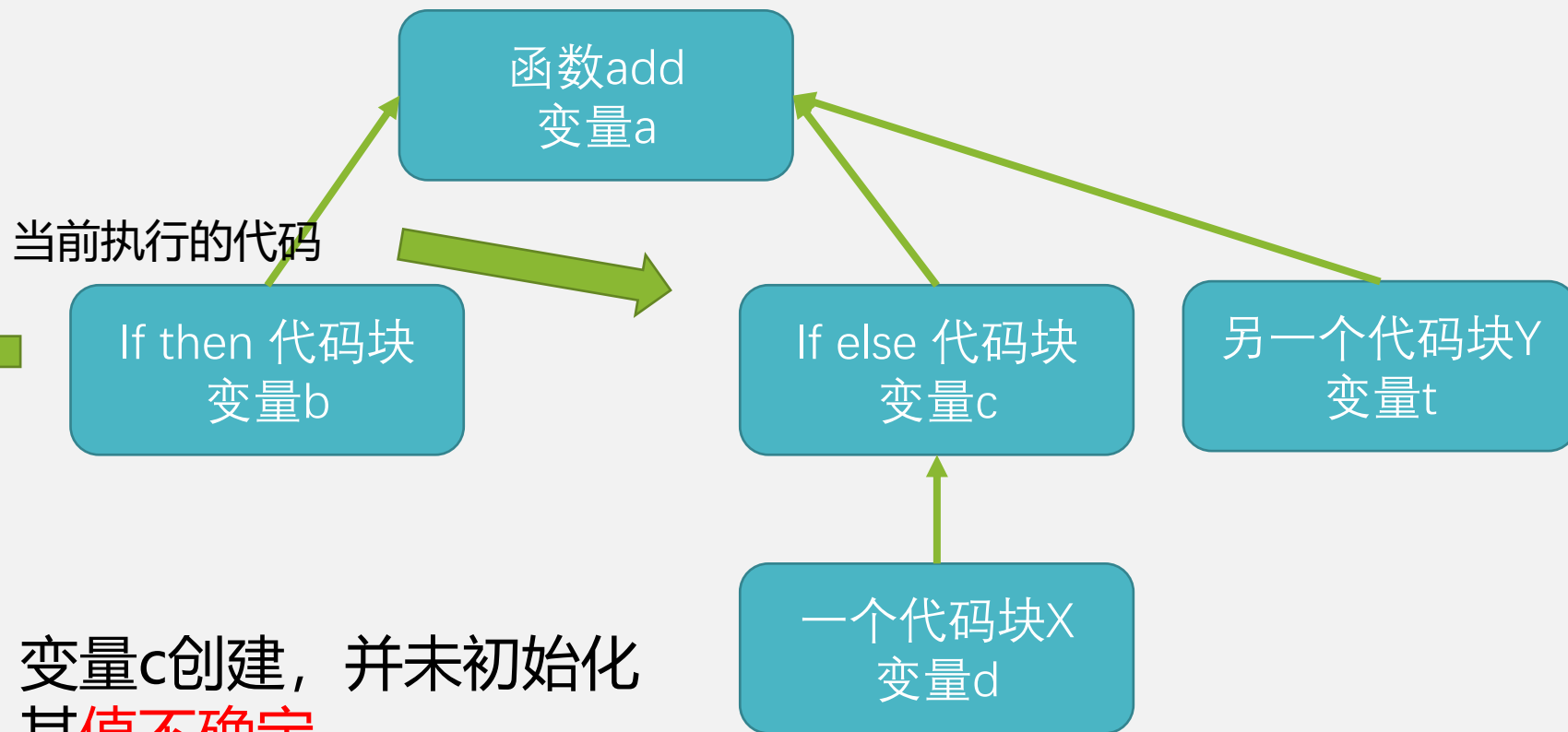




变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c;  
        {  
            int d = a;  
        }  
        c = a;  
    }  
    a = 9;  
    {  
        int t = a;  
    }  
}
```



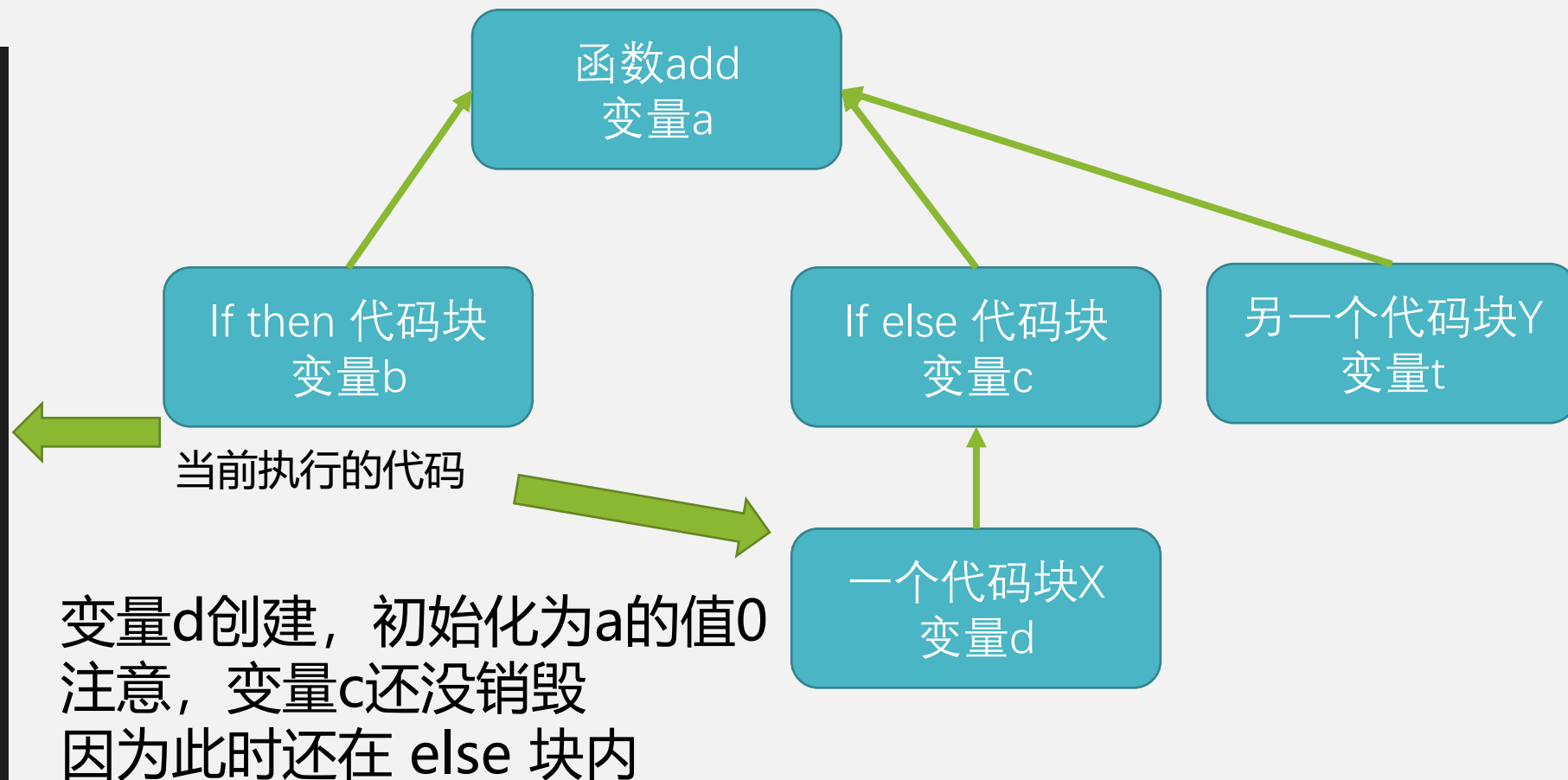
变量c创建，并未初始化
其值不确定。
注意，变量a不会销毁
因为此时还在函数块内



变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c;  
        {  
            int d = a;  
        }  
        c = a;  
    }  
    a = 9;  
    {  
        int t = a;  
    }  
}
```

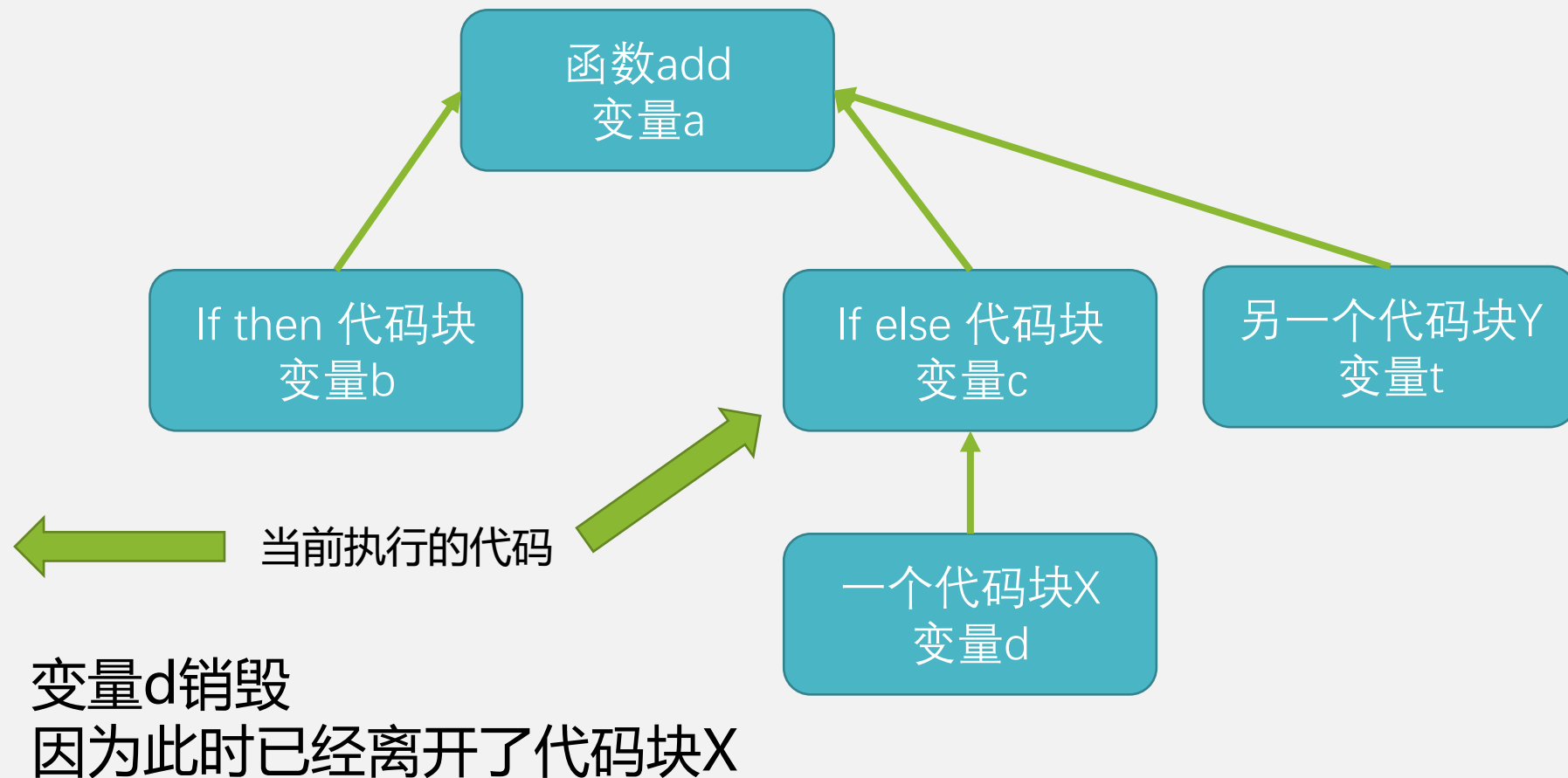




变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c;  
        {  
            int d = a;  
        }  
        c = a;  
    }  
    a = 9;  
    {  
        int t = a;  
    }  
}
```

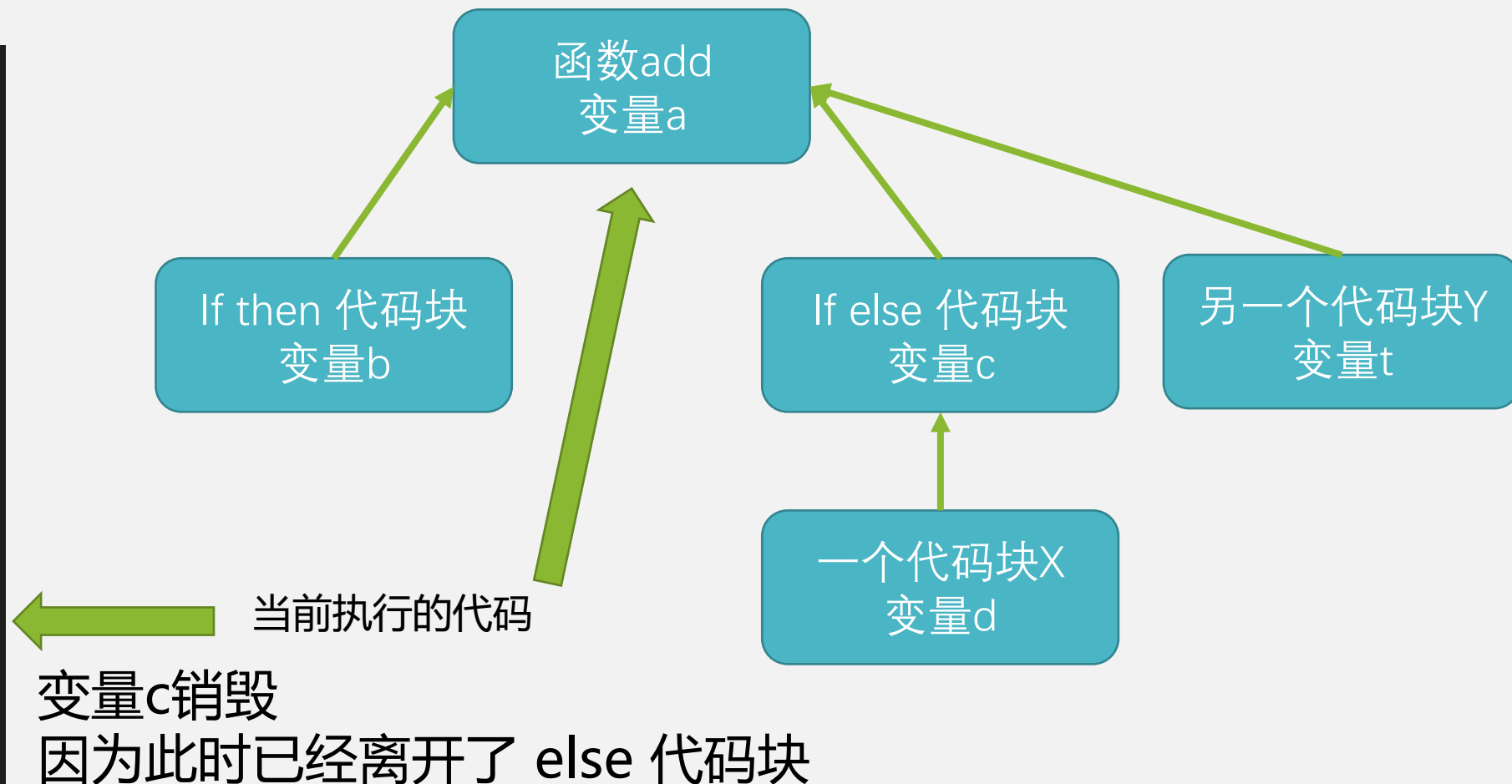




变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c;  
        {  
            int d = a;  
        }  
        c = a;  
    }  
    a = 9;  
    {  
        int t = a;  
    }  
}
```

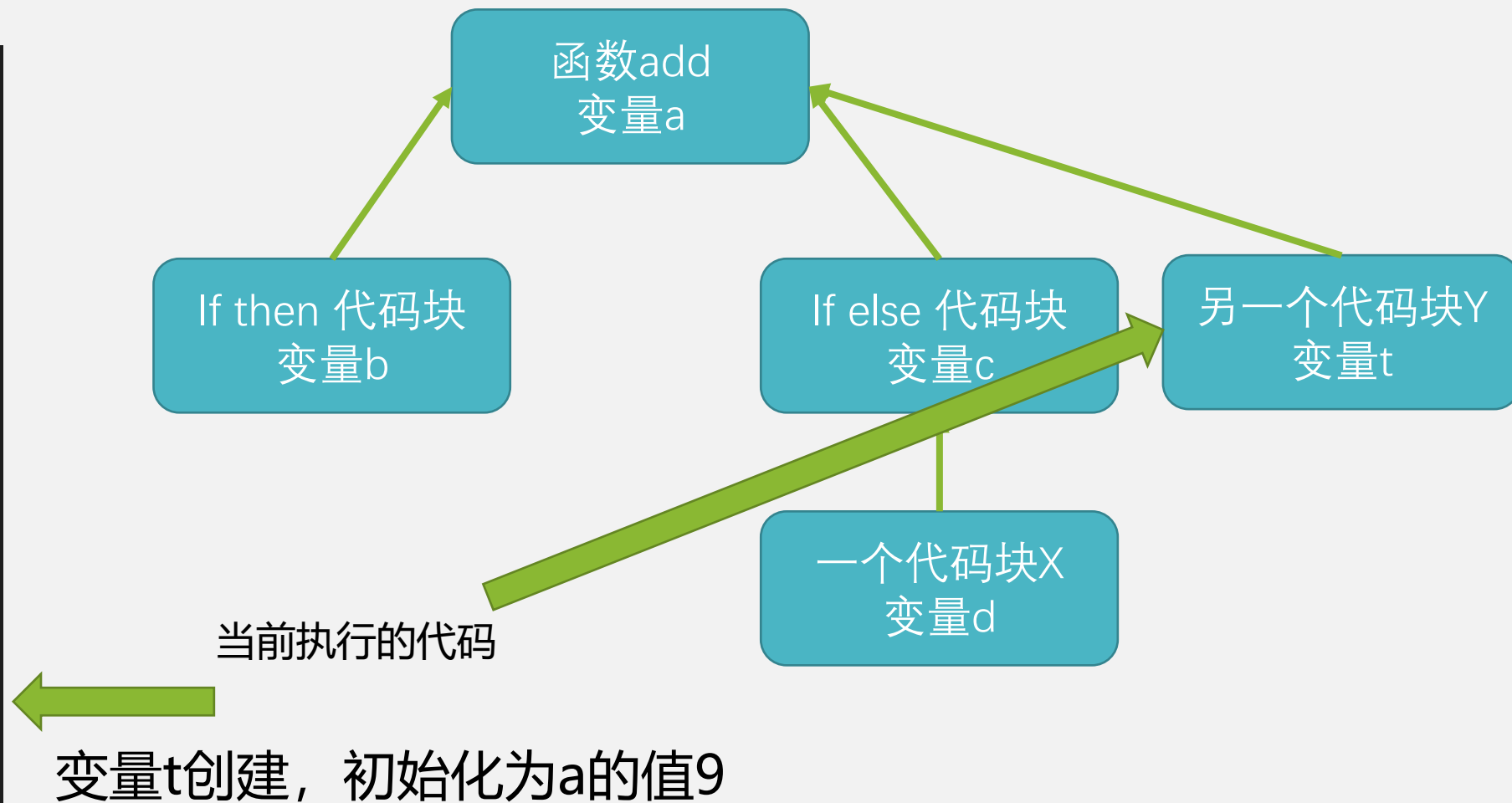




变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c;  
        {  
            int d = a;  
        }  
        c = a;  
    }  
    a = 9;  
    {  
        int t = a;  
    }  
}
```

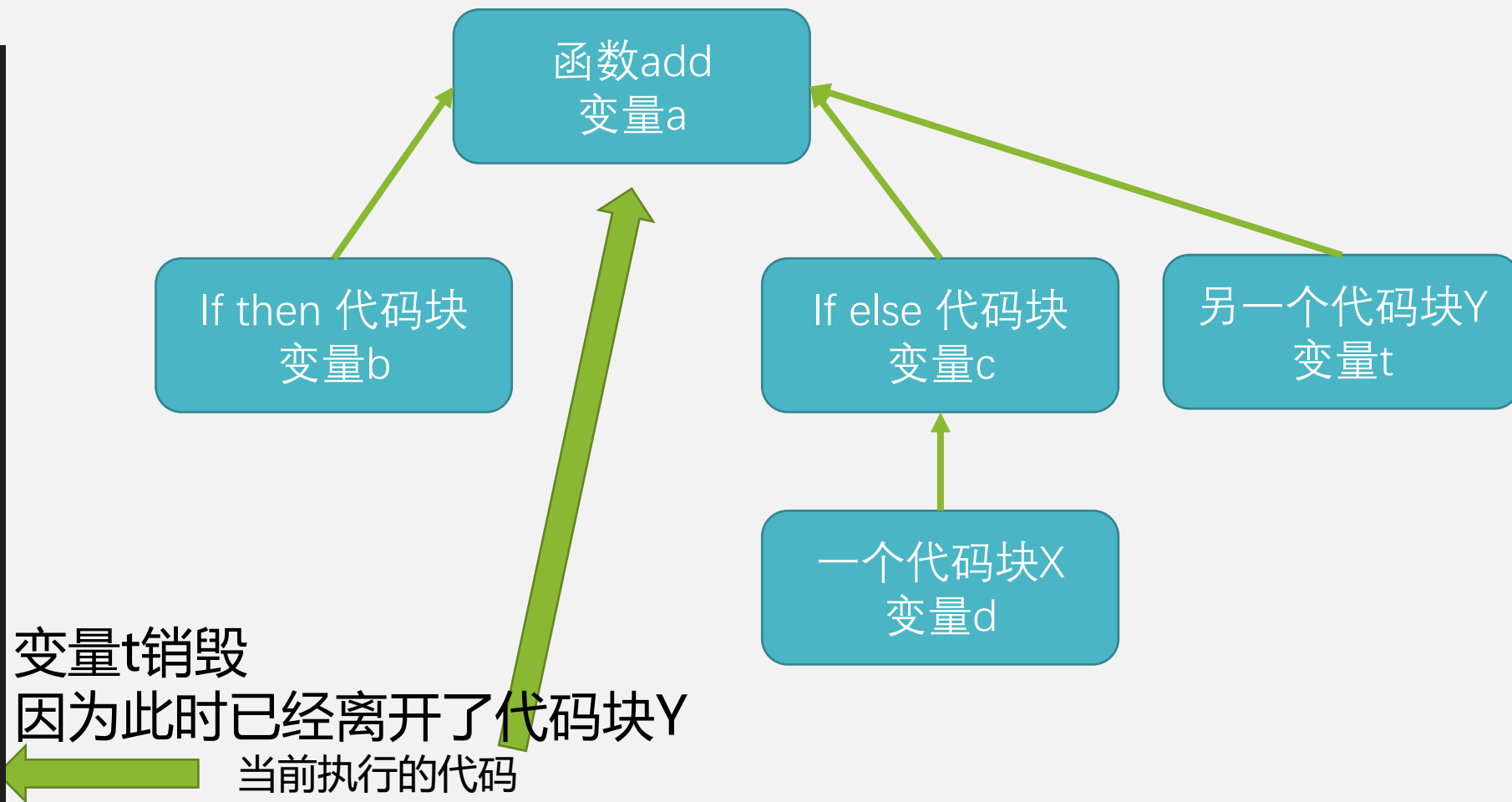




变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c;  
        {  
            int d = a;  
        }  
        c = a;  
    }  
    a = 9;  
    {  
        int t = a;  
    }  
}
```



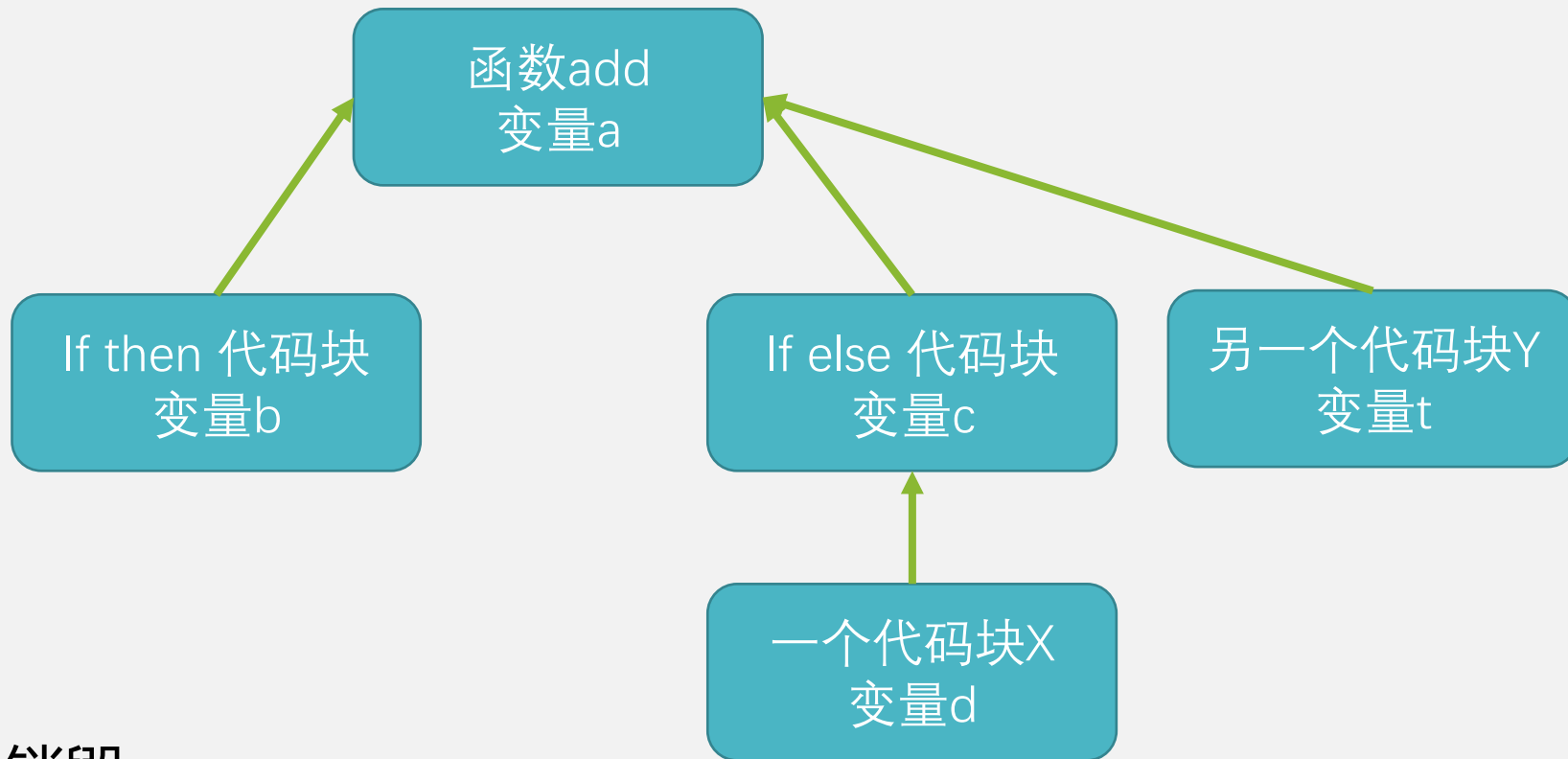


变量的作用域：局部变量

稍复杂的例子

```
void add(void){  
    int a = 0;  
    if (a == 1){  
        int b = 1;  
    }else{  
        int c;  
        {  
            int d = a;  
        }  
        c = a;  
    }  
    a = 9;  
    {  
        int t = a;  
    }  
}
```

变量a销毁
因为此时已经离开了函数块add



当前执行的代码

变量的作用域：全局变量

全局变量在函数外声明，一般在代码的顶部
在任何函数内都可以访问

```
#include <stdio.h>

int tot;

void add(void){
    int a = tot;
}

int main(){
    add();
    return 0;
}
```

全局变量tot

数据类型	初始值
int	0
char	'\0'
float	0
double	0
pointer	NULL

函数add里可以访问全局变量tot

全局变量会在程序初始时给予默认初始化（零）值
生命周期是整个程序的运行
直到程序结束才会被销毁



变量的作用域：形式参数

形式参数在函数参数列表声明
其性质和局部变量r一样，但会被调用函数的参数初始化

```
void add(int a, int b){  
    int r = 1;  
}  
  
int main(){  
    add(1, 2);  
    return 0;  
}
```

形式参数a,b，分别被初始化为1和2

在函数add内均可被访问，
生命周期持续整个函数执行过程，当函数执行完后才销毁



变量的作用域：变量同名

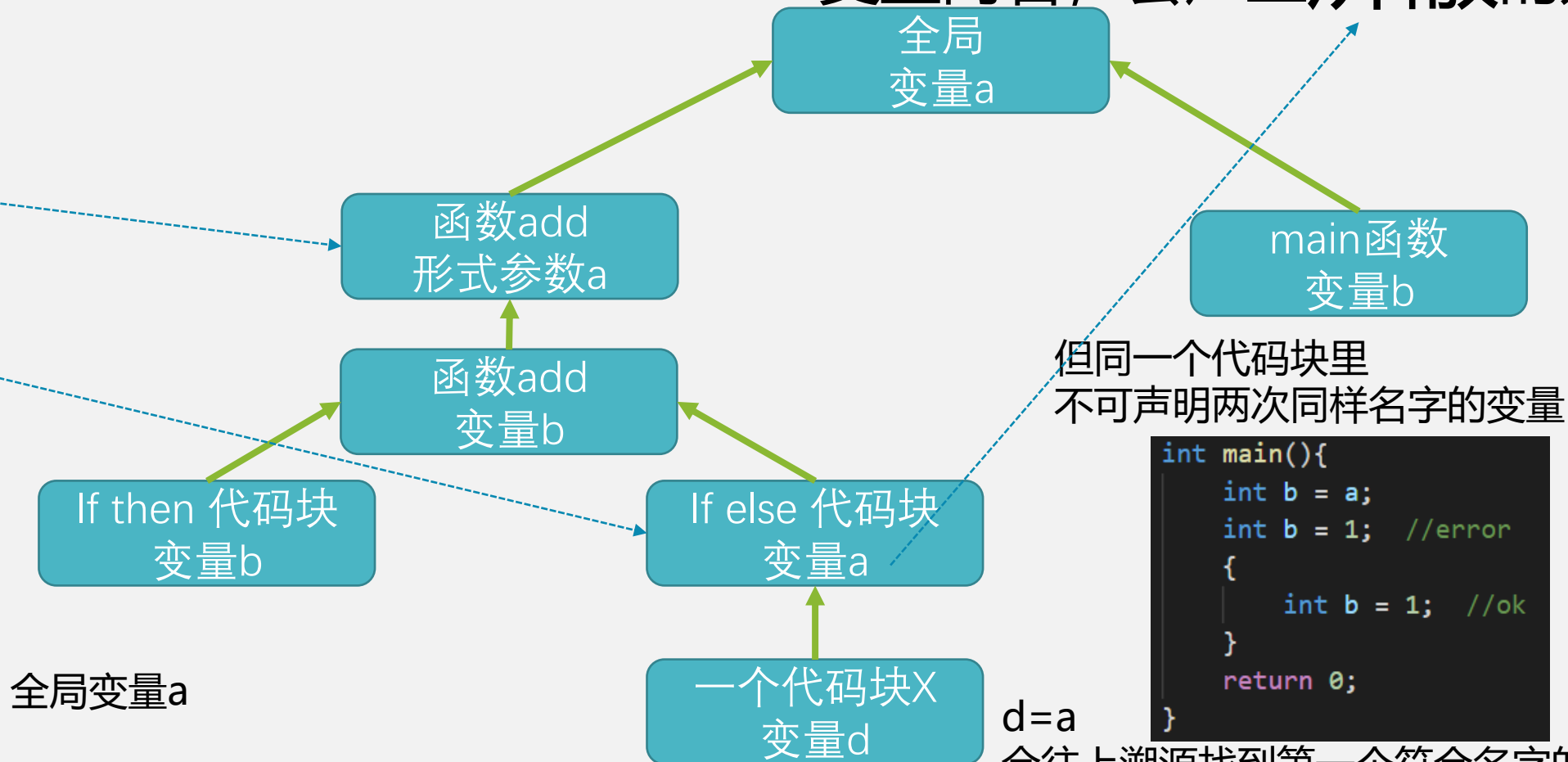
变量同名，会产生屏蔽的效果

```
#include <stdio.h>

int a;

void add(int a){
    int b = 0;
    if (a == 1){
        int b = 1;
    }else{
        int a = 2;
        {
            int d = a;
        }
    }
    a = 9;
}

int main(){
    add(0);
    int b = a;
    return 0;
}
```



```
int main(){
    int b = a;
    int b = 1; //error
    {
        int b = 1; //ok
    }
    return 0;
}
```




变量的作用域

位置	种类	初始化	生命周期
在函数或代码块里	局部变量 (local variables)	必须手动初始化	整个函数（块）
在所有函数外面	全局变量 (Global variables)	自动初始化为0	整个程序
在函数参数列表里	形式参数 (Formal parameters)	根据传输参数值 自动初始化	整个函数（块）



案例分析: dotcpp, 题目 1042: [编程入门]电报加密

```
#include<stdio.h>

/*****
 * encipher 编码字符。将字母变成其下一字母
 * (如'a'变成'b'.....'z'变成'a'其它字符不变)
 *
 * output: 编码后的字符
 * parameter:
 *     int 需要编码的字符
 *     int 偏移量 = 1
 * *****/
int encipher(int, int);
```

```
int main()
{
    char ch;
    // 读入一行电报输入
    scanf("%c",&ch);
    while (ch != '\n') {
        printf("%c",encipher(ch, 1));
        scanf("%c",&ch);
    }
    printf("\n");
    return 0;
}
```

请打开 ex-encipher.c 给出函数 encipher 的定义?



案例分析: dotcpp, 题目 1042: [编程入门]电报加密

解答:

```
int encipher(int c, int b) {  
    if (c >= 'a' && c <= 'z')  
        c = (c - 'a' + b) % 26 + 'a';  
    if (c >= 'A' && c <= 'Z')  
        c = (c - 'A' + b) % 26 + 'A';  
    return c;  
}
```

- 使用函数的好处
 - ...
- 一些基本概念
 - 函数是**初级表达式**(primary expression)
 - 函数声明的参数叫**形参**(form parameter)
 - 函数调用输入的参数叫**实参**(argument)
 - 当实参与形参不匹配时, 会自动**强制转换**
 - 右值表达式等级低于 int 的整数, 会自动**整数提升**



函数 (1) - 课后练习

Leetcode-cn.com, 题目 7. 整数反转

(请课后研究数学类题目, 例如: 题目 9. 回文数)



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



编制人：课题组